Dangers Behind Access Control: Understanding and Exploiting Implicit Permissions in Kubernetes

Nanzi Yang*
Xidian University
Xi'an, China
University of Minnesota
Minneapolis, Minnesota, USA

Xingyu Liu
Xidian University
Xi'an, China

Wenbo Shen
Zhejiang University
Hangzhou, China

Jinku Li†
Xidian University
Xi'an, China

Kangjie Lu
University of Minnesota
Minneapolis, Minnesota, USA

Abstract

As the de-facto standard for container orchestration, Kubernetes is extensively adopted by numerous companies and cloud vendors, making its security critical. In this paper, we define a new attack surface called *implicit permission*: The execution of explicitly granted permissions in Kubernetes dynamically leads to implicit operations on other resources, enabling new permissions beyond the explicitly granted ones. Such implicit permissions create security vulnerabilities that attackers can exploit to compromise an entire cluster.

Automatically identifying implicit permissions is challenging due to implicit relation reasoning and dynamic behaviors across diverse components of Kubernetes. To address that, we devise a systematic approach that combines static analysis techniques with the advanced capabilities of the large language model (LLM, e.g., GPT-4.5). Initially, we develop a static analysis to identify all Kubernetes resources. Building on this, we use static analysis to identify all explicit permissions for each resource. Finally, by combining the semantic reasoning capabilities of LLMs with the pattern-based precision of static analysis, we reason about what explicit permissions may dynamically lead to implicit permissions through complex interactions and uncover 593 implicit permissions derived from explicit permissions. We use the implicit permission references as insights to identify potential risks of CNCF projects and applications provided by the top four cloud vendors. With responsible disclosure, we obtain five new CVEs, six acknowledgments of cloud vendors, and a bounty awarded by Google. These acknowledgments underlie the practical impact of our attack.

CCS Concepts

• **Security and privacy → Distributed systems security**.

Keywords

Kubernetes, Access Control, Implicit Permission

ACM Reference Format:
Nanzi Yang, Xingyu Liu, Wenbo Shen, Jinku Li, and Kangjie Lu. 2025. Dangers Behind Access Control: Understanding and Exploiting Implicit Permissions in Kubernetes. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25), October 13–17, 2025, Taipei.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3719027.3765106

1 Introduction

As the de-facto standard for container orchestration, Kubernetes automates deployment, scaling, and management of containerized applications (or *apps*) [2]. The CNCF (Cloud Native Computing Foundation) 2023 survey reports that 84% of users are using or evaluating Kubernetes in their production environments [38]. The top four cloud vendors — Google, Amazon, Microsoft Azure, and Alibaba Cloud — offer managed Kubernetes services [22, 23, 26, 31], which are widely adopted by companies such as Shopify, GoDaddy, the NBA Stats Team, and CIBI.

Despite its efficiency, Kubernetes is prone to security issues, especially considering its complex access control system. Specifically, Kubernetes employs Role-Based Access Control (RBAC) to regulate how apps interact with cluster resources [20]. Recent studies [21, 51, 57] and tools [25, 32–36] have focused on identifying risky explicit permissions–whether explicit permissions are excessive in static RBAC policies. Our study reveals that an explicit permission brings dynamic and cascading effects during the execution, which are not specified by vendors and not understood by users. This leaves a critical gap in quantifying how permissions can expand their influence beyond their initial explicit configurations in real-world Kubernetes environments.

Systematically analyzing these permission behaviors in container orchestration systems carries two fundamental research challenges. *First*, it is challenging to abstract and define this phenomenon beyond ad-hoc cases. While Kubernetes' official documentation vaguely describes a phenomenon related to secrets [18] (i.e., *"creating a Pod in a namespace can use that access to read any Secret in*

*The majority of this work was performed during the author's affiliation with Xidian University. The author is currently at the University of Minnesota.
†Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CCS '25, Taipei
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1525-9/2025/10
https://doi.org/10.1145/3719027.3765106

that namespace"), it only provides fragmentary descriptions without establishing clear terminological definitions of the underlying security implications.

Second, even after establishing a clear definition, it is challenging to systematically identify these implicit permissions. While a specific permission reference [18] can be abstracted(i.e., *create Pod→get Secret*), there is no systematic approach to identify all these references in Kubernetes. These references arise not from simple misconfiguration but from dynamic privilege propagation triggered by the legitimate use of explicit permissions. These indirect effects emerge from the complex, component-driven behaviors of Kubernetes.

Prior research in other domains has explored specific cases of permission expansion beyond the initial explicit permissions [41, 42, 47, 48, 55, 56]. Permission expansion is a relevant but different problem from implicit permission. More specifically, these studies primarily focus on analyzing how granted explicit permissions ($p \in P_{\text{granted}}$) can lead to additional explicit permissions ($q \in P_{\text{granted}^*}$) through specific operations, such as permission re-delegation via IPC or Android custom permissions. In essence, both p and q remain within the scope of different explicit permission sets, where permission expansion explicitly bridges the two. As a result, permission expansion alone does not fully capture the complexity of real-world container orchestrators.

To solve the first challenge, in Kubernetes, we observe that some permissions exhibit dynamic behaviors, where a single granted permission may implicitly enable additional actions without directly transitioning between explicit permissions. This phenomenon introduces a new attack surface that we term *implicit permissions*. In Kubernetes, an explicit permission ($p \in P_{\text{granted}}$) might implicitly enable an operation on other resources, whose semantics are equivalent to new permission ($q \in P_{\text{implicit}}$), even though q was never explicitly granted in any granted explicit permission sets ($P_{\text{granted}} \cup P_{\text{granted}^*}$). This phenomenon differs from permission expansion because the operation itself inherently represents additional privileges in dynamic workflows rather than merely delegating or expanding existing explicit permissions.

For example, Service Account tokens in Kubernetes are stored and mounted as Secret resources, and accessing a service account token is semantically equivalent to accessing the underlying Secret. If a user is granted explicit permission: patch DaemonSet (p), due to the way Kubernetes handles DaemonSet patching, a high-privilege service account (e.g., `cluster-admin`) can be mounted and obtained by the DaemonSet, thereby enabling the attacker to get the associated Secrets and take over the whole cluster. Here the P_{implicit} = {`get Secrets`}. Such permission is not explicitly granted but arises dynamically as a cascading effect. We define such unintended but system-enabled capabilities as **implicit permissions**. When attackers exploit them to perform malicious actions beyond their granted privileges, we refer to such cases as **implicit permission attacks**.

To solve the second research challenge, we design and implement a systematic analysis tool to identify implicit permissions in Kubernetes. The tool is implemented through a three-step methodology: (1) identifying all resources, (2) extracting their explicit permissions, and (3) reasoning about implicit permission references through hybrid static and semantic analysis. Implementing this systematic approach involves difficulties in each step:

The first step is to identify all Kubernetes resources accurately. Kubernetes uses a resource-centric access control mechanism. Any unaccounted resource would leave a blind spot in the related explicit permission and following implicit permission analysis. Our key observation is that Kubernetes uses storage objects as metadata structures to record a resource's configuration and state. Each resource must register its storage object before it can be fully utilized. By locating these storage objects from the source code, we can identify all resources and collect all 116 resources with no omissions or misidentifications.

The second step is to extract all explicit permissions for each resource comprehensively. As mentioned before, every implicit permission is ultimately derived from an explicit one, and ensuring that all explicit permissions for each resource are captured helps avoid any blind spots. We find that once a storage object is registered, the system also registers the resource's operation methods, where each method corresponds to an explicit permission. Therefore, by extracting these methods from each resource, we can systematically extract all 635 explicit permissions without any being overlooked or misidentified.

The third step is to identify implicit permission references behind explicit ones. Theoretically, this can be conceptualized as a graph analysis problem, where nodes represent permissions and edges represent permission propagation relationships. However, it is hard to accurately construct these edges via static analysis alone due to two characteristics: implicit relation reasoning and dynamic behaviors across diverse components of Kubernetes. While static analysis excels at precise detection based on pattern-based rules, the complex characteristics of implicit permissions make it hard to capture all cases with limited rules.

To address this, we adopt a *hybrid decision-making* that combines the semantic reasoning capabilities of large language models (LLMs) with the structural precision of static analysis. LLMs excel at contextual understanding and domain-specific reasoning without pre-defined rules, enabling them to infer permission propagation patterns that are hard to capture with static rules. Complementarily, static analysis is integrated to handle deterministic, pattern-based cases that benefit from rule-based precision. Using this combined approach, we successfully identified 593 and triggered 328 implicit permission references. We further evaluated the contributions of each component through an ablation study, which confirms the complementary roles of LLM-based reasoning and static analysis.

To evaluate the real security impact of implicit permission attacks, we deploy all applications from the CNCF project lists and application marketplaces [24, 27, 30, 53] of the top four cloud vendors (i.e., Google, Amazon, Microsoft Azure, and Alibaba Cloud). We then use the analysis results from our approach as insights to identify real attacks. All experiments and attacks in this paper are conducted on dedicated clusters, which we use solely. As a result, we find multiple potential risks in these environments and report them through proper channels. Notably, five CNCF projects gave us CVE numbers for awarding our efforts. Google gave us five acknowledgments with one security bounty of $3133.70 to award our findings. Azure also awarded us one acknowledgment. These

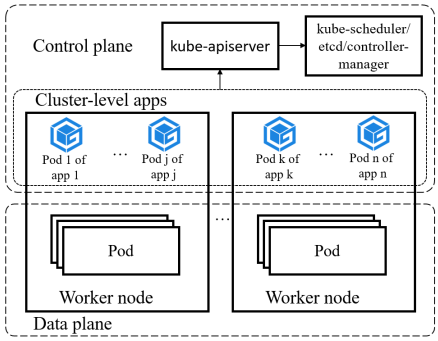


Figure 1: The overview of Kubernetes architecture.

```
Name:         default-kanister-operator-edit-role
...
Role:
  Kind:  ClusterRole
  Name:  edit
Subjects:
  Kind            Name                  Namespace
  ----            ----                  ---------
  ServiceAccount default-kanister-operator default
```

(a) The ClusterRoleBinding related to the *kanister-operator*.

```
Name:         edit
...
PolicyRule:
  Resources  Resource Names    Verbs
  ---------  --------------    -----
  ...
  daemonsets       []      [patch update ...]
  ...
```

(b) The `edit` ClusterRole used by kanister.

Figure 2: Details of related RBAC items.

results confirm that our approach not only identifies implicit permissions accurately but also highlights the approach's practical effectiveness in identifying real issues.

Overall, our major contributions are listed as follows:

New attack surface: We define a new attack surface in Kubernetes called *implicit permissions*. We further propose implicit permission attacks to demonstrate that these implicit permissions can be exploited to launch severe attacks.

Systematic analysis: We propose a combined approach that integrates static analysis with the LLM's capabilities for permission analysis. For broader validation, we comprehensively open-source our complete tool implementation, datasets, and identified results at: https://doi.org/10.5281/zenodo.16999749.

Practical impact: Based on our analysis, we identify multiple vulnerabilities in Kubernetes applications of CNCF projects and the top four cloud vendors. We have been awarded five new CVEs, acknowledgments of cloud vendors, and a bounty from Google.

2 Background

2.1 Kubernetes Architecture

Kubernetes [2] provides a framework for creating and managing containers at scale. As shown in Figure 1, a Kubernetes cluster comprises a control plane and a data plane. The control plane is responsible for managing and maintaining the cluster, which contains the Kubernetes components and cluster-level applications. In contrast, the data plane handles the actual user workloads running on multiple worker nodes managed by the control plane.

In the control plane, several Kubernetes components (including kube-apiserver, kube-scheduler, etcd, and controller-manager) maintain the cluster's functionalities. For example, the API Server acts as an external reference monitor, evaluating all API requests sent by containers to interact with multiple resources based on access control policies [14]. Once an incoming request is approved, the API Server may trigger a series of internal operations involving other components like Schedulers and Controllers. These components coordinate to achieve the desired configuration by manipulating related resources. For example, a request to update a Deployment [3] might result in the Controller creating or deleting Pods to maintain the desired replica count [8]. These internal operations may lead to actions that extend beyond the originally granted permissions, creating potential security risks when internal mechanisms trigger cascading effects.

In addition, the control plane typically hosts multiple cluster-level apps that enhance cluster functionality and offer essential services. These apps, maintained by diverse vendors and communities, vary from monitoring tools to security enhancements. Specifically, the CNCF hosts a broad array of 182 projects that are extensively utilized across various companies [39, 40]. Besides, major cloud vendors like Google, Amazon, Microsoft, and Alibaba enhance their managed Kubernetes services with apps in their marketplaces. Cluster-level apps play a vital role in enhancing Kubernetes clusters by interacting with various resources to provide critical functionalities. These interactions often rely on permissions that are explicitly configured. However, current security audits conducted by experts only focus on managing explicitly granted permissions, with no consideration for the implicit permissions that arise from Kubernetes' internal workflows. This oversight increases the complexity of permission management and introduces significant security risks, motivating us to systematically analyze the security implications of implicit permissions in Kubernetes clusters.

2.2 Role-Based Access Control

Kubernetes implements role-based access control (RBAC) to manage access permissions in a resource-centric manner [20]. In this system, permissions are directly tied to Kubernetes API operations, meaning that each RBAC rule defines specific API actions that users or applications are allowed to perform on certain resources. When containers attempt to access resources, the kube-apiserver processes these requests, allowing only those that comply with the explicitly granted RBAC permissions.

Kubernetes explicit permissions consist of two components: resources and verbs. Verbs define the actions that can be performed on a specific resource, and each resource supports multiple verbs. Therefore, the total number of permissions significantly exceeds the number of resources. For example, a single DaemonSet resource supports multiple verbs (e.g., create, get, list, delete, watch, etc.) [10]. This design philosophy results in a significantly higher number

of permissions than resources, as each resource-verb combination constitutes a separate permission.

However, explicit permissions are only part of the story. Actions allowed by explicit permissions involve additional internal workflows in Kubernetes, where Kubernetes components generate secondary API requests to fulfill the original operation. These subsequent API requests that operate on additional resources can be regarded as implicit permission as they grant access to resources not covered by explicit ones. Such implicit permissions are not explicitly configured in RBAC policies but arise from Kubernetes' internal mechanisms, posing hidden security risks.

For example, Kanister [44], a CNCF sandbox project, includes a component named `default-kanister-operator`, which uses a service account called `kanister-operator-sa`. This service account is bound with an `edit` ClusterRole (shown in Figure 2a) with explicit permission shown in Figure 2b. Specifically, the ClusterRole includes a "patch/update" verbs of DaemonSet resources (line 7 in Figure 2b).

While this explicit permission configuration appears secure from a static perspective, it interacts with Kubernetes' internal control mechanisms in non-obvious ways and results in unintended implicit permissions. The `patch` or `update` for DaemonSet may cause Kubernetes' internal DaemonSet controller to reconfigure pod templates with specific service accounts. These service accounts' tokens are automatically mounted and stored as Secrets resources in the newly-created Pod. Thus, seemingly limited explicit permission can lead to implicit access to high-privilege Secrets resources. We provide more details in §3.3.

3 Motivation

In this section, we first present the threat model and assumptions of our work. Then we describe the current status and weak spots of Kubernetes permission analysis. After that, we introduce the concept of implicit permissions and use a real example to illustrate the details.

3.1 Threat Model

In this paper, we consider two threat models:

Model I: node compromise: In this model, we assume that an attacker can compromise a worker node via escaping from a workload pod. The attacker aims to exploit the dangerous implicit permissions of Kubernetes applications on the same node and take over the cluster. This model reflects a common real-world scenario: user workloads typically run in containers with limited permissions, while third-party Kubernetes applications often require elevated cluster-level permissions for their functionality. For example, an attacker could exploit well-documented container escape vulnerabilities [54] such as CVE-2019-5736 (runC escape) to break container isolation and gain worker node access. Once node access is gained, the attacker can exploit the implicit permissions carried by Kubernetes apps on the same node to achieve cluster-level privilege escalation. This model is also accepted by multiple prior works [21, 51, 57].

Model II: direct pod compromise: Similar to the approach established in EPScan [43], we also consider scenarios where an attacker can directly compromise a pod with elevated permissions and aims to exploit implicit capabilities. While this represents a different initial assumption from Model I, our core contribution centers on understanding how explicit permissions implicitly enable additional capabilities. Any pod with elevated permissions can serve as a starting point for demonstrating implicit permission attacks. For example, CVE-2025-1974 [13] demonstrates this model: an attacker with access to any pod in the cluster can directly send malicious AdmissionReview requests to the ingress-nginx controller's admission webhook, achieving configuration injection, remote code execution, and compromising the Ingress pod. After that, the attacker can abuse its implicit permissions and take over the whole cluster.

3.2 Existing Research and the Differences

To address research challenge 1 (i.e., defining what constitutes implicit permissions in container orchestration environments), we first establish the conceptual foundations by examining how implicit permissions differ from existing Kubernetes excessive permission works. After that, we propose our definition.

Existing excessive permission works: In the Kubernetes scenario, excessive permission [21, 43, 51, 57] is the state-of-the-art research that discusses the potential risks carried by access control. Excessive permissions are those permissions allowed by RBAC policies explicitly, which are unnecessary for Kubernetes apps' normal functionalities. They may be abused to make privilege escalation risks.

Definition: excessive permissions. Let P_{granted} be the set of explicitly granted permissions to an app S, and let $P_{\text{required}} \subseteq P_{\text{granted}}$ be the minimal permission set required for S's normal functionalities. Formally, the excessive permissions can be defined using the set difference operation, denoted by the symbol "\", as follows:

$$P_{\text{excessive}} = P_{\text{granted}} \setminus P_{\text{required}} = \{p \mid p \in P_{\text{granted}}, p \notin P_{\text{required}}\}$$

This means that $P_{\text{excessive}}$ contains precisely those permissions that have been explicitly granted but are not essential to the app's basic functionality.

While this definition captures statically redundant permissions, these works are unaware of a critical aspect: if one critical permission is essential for the normal operation of a Kubernetes application, then it is not excessive and is in ($p \in P_{\text{required}}$). Despite this necessity, attackers can still exploit the permission to escalate privileges, demonstrating that the fundamental issue is not whether the permission is excessive.

Our proposed implicit permission: From our perspective, granting explicit permission means allowing an operation on one specific resource. However, due to Kubernetes' internal automation and scheduling mechanisms, this operation frequently leads to another operation on other resources, which is not explicitly allowed by explicit RBAC rules. Operations on other resources can be semantically equivalent to new permissions beyond the explicit one, which serves as the underlying enabler of the attacking scenario.

Definition: implicit permissions. Formally, let P be the set of all Kubernetes permissions, and let P_{granted} be a given set of explicit permissions. Let $R \subseteq P \times P$ be a relation where $(p, q) \in R$ indicates that exercising an operation permitted by explicit permission p implicitly enables another operation represented by permission q

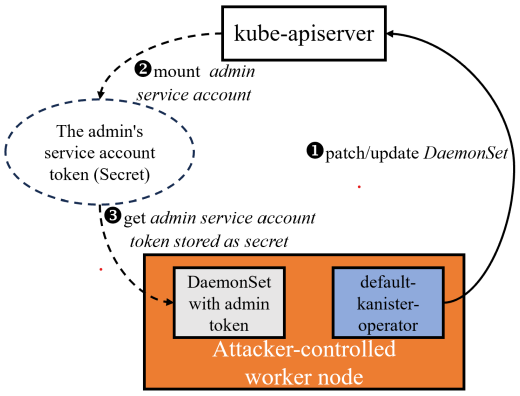


Figure 3: Implicit permission attack via Kanister.

through system mechanisms. The implicit permission set is:

$$P_{\text{implicit}} = \bigcup_{p \in P_{\text{granted}}} \{q \mid (p, q) \in R\} \setminus P_{\text{granted}}$$

This means that even though q is not explicitly granted in RBAC policies, it becomes accessible due to Kubernetes' built-in automation, such as controllers, schedulers, etc.

For example, consider a scenario where the explicit permission `patch DaemonSet` is granted and required for an application's normal functionalities, thus `patch DaemonSet` $\in P_{\text{required}}$ and the excessive permission set is empty. The attacker can abuse this to modify a DaemonSet and mount a high-privilege ServiceAccount. As SA tokens are stored as Secrets and mounted into pods, this implicitly grants access to Secret resources. Although `get Secret` is not explicitly granted, the ability to extract a token from a pod created via a modified DaemonSet is semantically equivalent to having `get Secret` on that token's Secret object. Formally, the implicit permission propagation can be represented as:

$$(\texttt{patch DaemonSet}, \texttt{get Secret}) \in R; \quad P_{\text{implicit}} = \{\texttt{get Secret}\}$$

Note that the `get Secret` permission is beyond the scope of the explicit permission set P_{granted} (`patch DaemonSet`), indicating that it is an implicit permission enabled by Kubernetes' internal mechanisms. Crucially, while $P_{\text{excessive}}$ focuses on eliminating unnecessary explicit permissions, it fails to capture risks arising from permission propagation. In contrast, P_{implicit} reveals hidden attack surfaces that remain even when permissions are strictly minimized.

3.3 Real World Case: Implicit Permission Attack via Kanister

To further validate the implicit permission patterns observed by us, we discuss a real-world deployment scenario where implicit permissions lead to a privilege escalation attack. Kanister [44] is a CNCF project that provides data management tools for Kubernetes. It includes a component called `default-kanister-operator`, which runs in the cluster with a bound ServiceAccount named `default-kanister-operator`. This ServiceAccount is associated with a ClusterRole that explicitly grants it the `patch` and `update` verbs on DaemonSet resources (as shown in Figure 2b).

Under normal circumstances, these permissions are necessary for Kanister to perform update operations on managed DaemonSets. However, if an attacker compromises the node running the `default-kanister-operator` pod, they can exploit this legitimate `patch DaemonSet` permission to launch an implicit privilege escalation attack. Specifically, the attacker can modify an existing DaemonSet's pod template to specify a high-privilege ServiceAccount, such as `cluster-admin` (❶ in Figure 3). Due to the behavior of the Kubernetes DaemonSet controller, new pods created from this DaemonSet will automatically mount the token of the specified ServiceAccount (❷ in Figure 3). Since ServiceAccount tokens are stored as Kubernetes Secrets, the attacker—by accessing the filesystem of the compromised node—can extract the token from the newly configured pod of DaemonSet (❸ in Figure 3). This token effectively grants the attacker full cluster-admin privileges, allowing them to bypass RBAC policies and gain control over the entire cluster. Notably, the attacker was never explicitly granted `get Secrets` permission. Instead, this powerful capability emerged implicitly from the originally benign-looking `patch DaemonSet` permission.

We reported the issue to the project maintainers via GitHub's security disclosure channel. The CNA, GitHub Inc., has assigned it the identifier CVE-2024-43403, with a CVSS v3.1 base score of 8.8 (HIGH). This real-world case demonstrates how even non-critical explicit permissions can become dangerous vectors for privilege escalation when viewed through the lens of implicit permissions. It highlights the importance of analyzing not only what explicit permissions are granted but also how those permissions may propagate implicitly within the system.

4 Systematic Analysis Framework

In this section, we first introduce the limitations of existing Kubernetes permission analysis tools. After that, to solve the second research challenge, we discuss our approach and illustrate the details.

4.1 Limitations of Existing Tools

There are multiple existing tools [25, 32–36] for identifying risky Kubernetes permissions. Typically, they take apps' permission configurations as input, perform a rule-based check for known risky permissions, and output vague security warnings when predefined risky permissions are found. For example, when apps are granted the "update" verb on DaemonSets, the krane tool [32] would flag this risky permission and output *"Roles/ClusterRoles allowing to update a potentially malicious daemonset. This might be dangerous."* [45].

However, these works have several shortcomings. First, these works have limited scope, as they are restricted to permissions explicitly defined in apps' RBAC configurations and can not analyze all Kubernetes permissions. Second, they rely on pre-defined risk rules to flag only explicit permissions and cannot infer privilege escalation paths through implicit permission propagation. Third, their risk identification is case-specific and dependent on security experts rather than a systematic approach to uncover all implicit permissions. As we discussed before, they only flag the explicit `update DaemonSet` and fail to detect implicit `get Secrets` that arise when a user updates a DaemonSet and specifies a high-privilege ServiceAccount in the pod template.

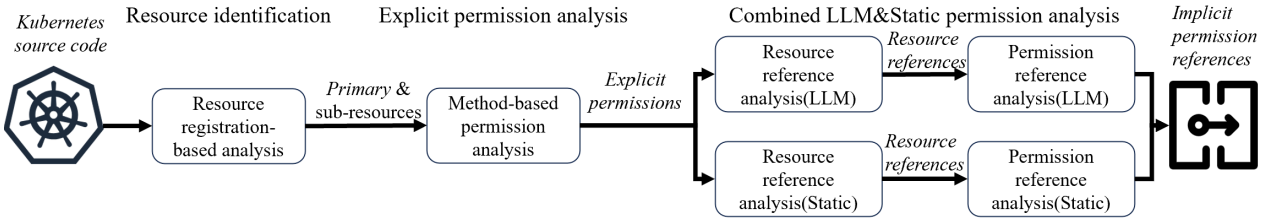


Figure 4: The architecture of our approach.

4.2 Approach Overview

To understand and analyze implicit permissions in Kubernetes, we need to go beyond RBAC configurations and examine how internal control-plane behaviors semantically propagate privileges. An explicit permission (e.g., patch DaemonSet) can implicitly enable sensitive operations (e.g., accessing Secret) even though such capabilities are not directly granted. These hidden privilege flows arise from dynamic system behaviors such as controller-driven pod creation, service account token mounting, and resource bindings. As a result, it is challenging to identify implicit permissions, which is the second research challenge of our work.

To systematically uncover such implicit permission paths, our analysis needs to resolve three essential questions: (1) What resources exist in the system? (2) What explicit permissions apply to each resource? and (3) How do explicit permissions propagate to other resources through system behaviors?

Answering these questions involves three steps with difficulties. First, it is hard to identify all resources completely and accurately. A natural idea to solve this problem is to collect resources from the official documents. However, the documentation only maintains the major versions of core resources (e.g., 1.29.0, 1.30.0, 1.31.0, etc.) [5–7], lacking visibility into sub-version differences, which may still be used in real-world clusters. Relying solely on the documentation leads to a version-biased resource inventory.

Second, accurately identifying all explicit permissions is essential, as implicit permissions are derived from them. Any error or omission in explicit permission identification can lead to incorrect implicit permission analysis.

Third, it is hard to analyze the implicit permission references behind each explicit one. Implicit permission analysis is particularly complex due to two key characteristics: implicit relation reasoning and dynamic behaviors across components. For implicit permission reasoning, the implicit permissions are often semantically reasoned from the operations on resources behind the explicitly allowed ones. For dynamic behaviors, these relationships must be reasoned across the dynamic execution paths of multiple Kubernetes components.

For example, the modification of DaemonSet implicitly carries the relationships between a high-privilege service account token with the sensitive Secret resource. This implicit permission (e.g., `get Secrets`) is not obvious from the modification operation alone. Instead, when modifying a DaemonSet, the operation implicitly carries relationships with Secret resources through semantic dependencies (DaemonSet→ ServiceAccount → Secret mounting) and coordinated behaviors across multiple components (kube-apiserver → DaemonSet controller → Pod controller → Token controller). Although no explicit permission was granted to access Secrets, coordinated behaviors implicitly enable access to sensitive credentials.

Approach overview: We propose a systematic approach to identify implicit permissions by combining accurate resource-permission mapping with semantic reasoning behind each explicit permission. As shown in Figure 4, our system integrates three key techniques: *resource registration-based analysis*, *method-based permission analysis*, and a *combined LLM & static permission analysis* for implicit permission inference. Each technique is designed to address one of the difficulties introduced earlier.

To accurately identify resources, our key insight is that every Kubernetes resource—whether a primary or sub-resource—will be registered in the system before it becomes usable. By collecting and analyzing these registration patterns from the Kubernetes source code, we can construct a complete and version-accurate view of all resource types present in the environment (see §4.3).

To accurately identify explicit permissions carried by each resource, our key observation is that a set of methods is registered for each resource during initialization. These methods define allowed operations and correspond to explicit permissions (verb-resource pairs). By analyzing the method registration pattern, we can accurately recover the full set of explicit permissions (see §4.4).

Why LLM & Static Analysis? Implicit permissions arise because operations allowed by explicit permissions can affect the status of other resources. These status changes can be semantically equivalent to indirect operations on those other resources, thus constituting implicit permissions. Therefore, implicit permission analysis can essentially be viewed as a graph analysis problem, where nodes represent specific operational permissions on resources, and edges represent propagation relationships between permissions.

Traditional static analysis relies on clear and explicit rules to construct edges. However, implicit permission relationships often span multiple components and are encoded in complex internal control logic, making it challenging to enumerate all cases with predefined rules. Some of these relationships follow well-defined inter-procedural paths—for example, how a patched DaemonSet triggers pod creation—which can be accurately captured via static control flow analysis. However, other edges are less explicit and rely on domain-specific semantics, such as understanding that a mounted ServiceAccount token implies access to the corresponding Secret.

To address both cases, we employ a hybrid decision-making approach that combines the structural precision of static analysis with the semantic generalization of LLMs. While static analysis handles well-defined patterns, LLMs focus on inferring implicit

relationships that rely on domain-specific semantics and contextual understanding. We take the union of both results to make a comprehensive identification of implicit permission references.

For permission flows with well-structured code paths, we use static analysis to trace the paths between resource controllers and the internal functions responsible for handling other resource types. This allows us to construct propagation paths from explicit permissions to operations on additional resources, enabling the identification of certain cross-component implicit permission references (see §4.5.2).

For complex semantic dependencies that cannot be captured by static rules, we leverage LLMs to interpret resource relationships and contextual understanding carried by dynamic behaviors. LLMs help infer non-obvious permission implications by understanding resource semantics and usage intent based on domain knowledge.

Besides, to mitigate the well-known hallucination problem inherent in LLMs, we employ a structured two-step inference process (resource reference and permission reference analysis) with self-reflection mechanisms applied at each step to eliminate hallucinations and prevent cascading hallucinations, thereby enhancing result accuracy (see more details in §4.5.1).

```
1  func (p RESTStorageProvider) storage() () {
2    ...
3    // clusterroles
4    if resource := "clusterroles";
5  apiResourceConfigSource.ResourceEnabled() {
6      storage[resource] = clusterrolepolicybased.NewStorage()
7    }
8    // clusterrolebindings
9    if resource := "clusterrolebindings";
10 apiResourceConfigSource.ResourceEnabled() {
11   storage[resource] = clusterrolebindingpolicybased.NewStorage()
12   }
13   ...
14 }
```

(a) The registration of ClusterRole and ClusterRoleBinding.

```
1  func (p *legacyProvider) NewRESTStorage() () {
2      ...
3      if resource := "pods"; apiResourceConfigSource.ResourceEnabled() {
4          storage[resource] = podStorage.Pod
5          storage[resource+"/attach"] = podStorage.Attach
6          storage[resource+"/exec"] = podStorage.Exec
7          ...
8      }
9      ...
10 }
```

(b) The registration of Pod and related sub-resources.

Figure 5: Registration of Kubernetes resources.

4.3 Resource Identification

To achieve the first step, we propose *resource identification* based on source code. There are two types of Kubernetes resources. One is the primary resource, as the main objects managed by Kubernetes (e.g., Pods, Deployments, Jobs, etc.). In contrast, sub-resources are associated elements (e.g., pod/exec, deployment/status, etc.) that are linked to the primary resources but managed separately. We leverage the different characteristics of Kubernetes resource registration to identify these two types of resources accurately.

Identifying primary resources: To identify all primary resources in the Kubernetes codebase, our key observation is that each primary resource registers a storage object before it is used [16, 17, 19]. In Kubernetes, a storage object is a data structure responsible for managing the lifecycle of a resource. By focusing on the initialization and registration patterns of these storage objects, we can systematically uncover all primary resources.

In particular, our technique focuses on the conditional statements that check resource availability, typically structured as `if resource := "resourcename"`. This pattern is a key marker indicating the beginning of resource configuration. The subsequent initialization and registration steps, such as `storage[resource] = NewStorage(...)`, are part of the overall setup process. For example, the func storage() function shown in Figure 5a is a key part of the Kubernetes framework code in Go language, which creates a storage map containing ClusterRole and ClusterRoleBinding resources. It first declares and initializes a local variable named resource (lines 4 and 9), checks if the resource is enabled, and then sets a new storage object for the resource key in the map (lines 6 and 11). By pinpointing these conditional checks, we can effectively collect the names of all primary resources.

Identifying sub-resources: To identify sub-resources accurately, our key observation is that the registration process of sub-resources is inherently linked to their corresponding primary resources [16, 17, 19]. This connection provides a systematic way to trace and document all related sub-resources following the primary resource's registration.

Our technique for identifying sub-resources focuses on analyzing configuration sequences right after a primary resource is recognized. We search for specific lines where sub-resources are registered using a combination of the primary resource name and sub-resource identifier, such as "subResourceStorage[resource + "/subresourcename"] = NewSubResourceStorage(...)". This allows us to effectively catalog all associated sub-resources, ensuring a comprehensive inventory of Kubernetes resources. For example, the func NewRESTStorage() sets up storage objects for Pod and its sub-resources. It first registers the primary resource Pod by configuring the pod's storage object (lines 3 and 4). Subsequently, it registers sub-resources by associating keys combining pods with sub-resources "/attach" and "/exec" (lines 5 and 6). As a result, we can extract the names and associations of sub-resources in such storage object configurations.

4.4 Explicit Permission Analysis

We leverage the characteristics of mapping between Kubernetes resource methods and permissions to identify all explicit permissions. Our approach includes identifying explicit permissions for primary resources and their sub-resources.

Identifying explicit permissions of primary resources: To identify all explicit permissions of primary resources, our key observation is that all primary resources are linked to specific verbs through the methods of the corresponding storage object [9, 11, 12]. These methods define the operations that can be performed on the primary resource, providing a path to permission identification.

More specifically, for each primary resource, we extract the corresponding methods implemented in their storage objects and directly map these methods to the corresponding API verbs. Kubernetes typically aligns method names within storage objects to specific verbs, which represent the operations they perform. By analyzing these

```
1  func (p *legacyProvider) NewRESTStorage() () {
2      ...
3      if resource := "pods";
       ↪  apiResourceConfigSource.ResourceEnabled() {
4          storage[resource] = podStorage.Pod
5          storage[resource+"/attach"] = podStorage.Attach
6          storage[resource+"/status"] = podStorage.Status
7          ...
8      }
9      ...
10 }
11
12 type PodStorage struct {
13     Pod         *REST
14     Status      *StatusREST
15     Attach      *podrest.AttachREST
16     ...
17 }
18
19 type REST struct {
20     *generic registry.Store
21     ...
22 }
23
24 func (e *Store) Create(...) (...) {
25   ...
26 }
27
```

❶extract corresponding methods of Pod object

❷the Create method can be directly mapped to create verb of pod resource

(a) The Create method of pod resource.

```
1  type StatusREST struct {
2    store *genericregistry.Store
3  }
4
5  func (r *StatusREST) Get(...) (...) {
6    return r.store.Get(...)
7  }
8
9  type ExecREST struct {
10   Store      *genericregistry.Store
11   KubeletConn client.ConnectionInfoGetter
12 }
13
14 var upgradeableMethods = []string{"GET", "POST"}
15
16 func (r *ExecREST) ConnectMethods() []string {
17   return upgradeableMethods
18 }
19
20
```

①the Get method directly mapped to get verb of pods/status resource

❷HTTP methods can be mapped to get/create verb of pods/exec resource

❶return an array contains supported HTTP methods

(b) The methods of the pods/exec and pods/status sub-resources.

Figure 6: The methods of Pod and related sub-resources.

methods associated with the storage object, the explicit permissions for the primary resource can be systematically determined.

For example, as shown in Figure 6a, the Pod storage object is registered using the `NewRESTStorage` method in line 4. This storage object, specified as `PodStorage.Pod` in line 13, is a REST object that inherits methods from the generic `registry.Store` (❶ in Figure 6a). One of the key methods inherited is the `Create` method, which directly maps to the create verb for the Pod resource (❷ in Figure 6a). This mapping is crucial as it defines the explicit permission to create a Pod instance within the Kubernetes cluster. Through this detailed analysis, we can extract the verbs associated with the methods of primary resources like Pods, thereby identifying explicit permissions.

Identifying explicit permissions of sub-resources: To identify explicit permissions for sub-resources in Kubernetes, our observation is twofold. First, for some sub-resources, explicit permissions can still be identified using the same approach as primary resources. However, for other sub-resources, explicit permissions are indicated through the array of HTTP methods returned by a unified `ConnectMethods`, which indirectly links these methods to specific API verbs.

For each sub-resource, we start by extracting methods from its storage object to pinpoint those corresponding to explicit permissions. In cases involving a `ConnectMethods`, we extract the array of HTTP methods it returns. These methods are then mapped to specific permission verbs based on mappings found in Kubernetes source code [15]. Through this approach, we can systematically determine the explicit permissions for each sub-resource, whether they are defined directly or derived from the `ConnectMethods`.

For instance, as shown in Figure 6b, the pods/status sub-resource utilizes a StatusREST type object (line 1), equipped with a Get method. Consequently, this setup grants the pods/status the "get" permission (① in Figure 6b). In contrast, the pods/exec sub-resource is associated with a method called `ConnectMethods` (line 16). This method outputs an array named `upgradeableMethods`, comprising the GET and POST HTTP methods [15] (❶ in Figure 6b). These methods align with the "get" and "create" verbs applicable to pods/exec according to the map in Kubernetes source code (❷ in Figure 6b).

4.5 Implicit Permission Analysis

In our combined analysis architecture, LLM-based reasoning and static analysis are executed in parallel. Each component independently analyzes the same set of explicit permissions and attempts to identify associated implicit permission paths.

While our approach overview discussed in §4.2 presents static analysis before LLM-based reasoning, we begin this section with the LLM-based approach. This order reflects our empirical observation that the majority of implicit permission relationships arise from semantic dependencies and loosely coupled system behaviors, which cannot be captured by fixed structural patterns. Only a minority of permission flows follow well-defined procedural logic suitable for static analysis. As a result, we first describe how LLMs can identify general-purpose semantic implications before turning to how static analysis is applied to trace structurally well-defined cases. The results from both analyses are then merged to form a unified permission reference, which captures both well-defined and non-obvious implicit permission references.

4.5.1 LLM-based Analysis. We explore the logical reasoning capacities and the extensive pre-trained domain knowledge inherent in the large language model (i.e., GPT-4.5) to identify implicit permission references carried by explicit permissions. To ensure reliability and mitigate potential hallucinations, our LLM-based approach employs a structured two-stage process: resource reference analysis and permission reference analysis. For each stage, we provide LLM with few-shot examples from Table 1 as predefined concepts and prompt it to generate reasoning. For reasoning results of each stage, we enforce LLM to make binary Pass/Fail decisions; for failed cases, LLM self-corrects until passing. We apply self-reflection to resource/permission references sequentially, preventing cascading hallucinations. Besides, we provide only 10 example pairs while analyzing hundreds of permission combinations. This significant scale difference shows the LLM must generalize and infer implicit relationships rather than merely retrieve from provided examples.

Table 1: Training examples of primary and sub-resource permission references, Ref. means Reference.

Primary	Resource Ref.	Permission Ref.	Sub-resource	Resource Ref.	Permission Ref.
StatefulSet	StatefulSet → Pod	create StatefulSet → create Pod	pod/exec	pod/exec → secrets	create pod/exec → get secrets
Job	Job → Pod	create Job → create Pod	replicasets/scale	replicasets/scale → Pod	patch/update replicasets/scale → create/delete Pod
Daemonset	Daemonset → Pod	create Daemonset → create Pod	deployments/scale	deployments/scale → Pod	patch/update deployments/scale → create/delete Pod
Node	Node → Pod	patch Node → delete/create Pod	pods/eviction	pods/eviction → Pod	create pods/eviction → delete Pod
ClusterRoleBinding	ClusterRoleBinding → ClusterRole	create ClusterRoleBinding → * verbs of * resources	statefulsets/scale	statefulsets/scale → Pod	patch/update statefulsets/scale → create/delete Pod

Resources references analysis (LLM): For resource reference analysis, our key observation is that there exist dependency relationships among resource objects. These dependencies imply that actions performed on one resource can implicitly influence or trigger actions on another resource, forming the basis for subsequent permission reference analysis.

For example, as listed in Table 1, resources such as StatefulSet, Job, DaemonSet, and Deployment manage different sets of Pods. Explicit creations of these resources will implicitly create associated Pods. Similarly, sub-resources such as the replicasets/scale and statefulsets/scale manage the number of related Pods. Explicit modifications (e.g., patch/update) on these resources will implicitly create/delete associated Pods.

To systematically identify resource dependencies, we propose a resource reference analysis framework. The analysis begins by instructing the LLM to act as a Kubernetes expert. Through few-shot learning using resource reference examples from Table 1, the LLM gains a conceptual understanding of resource references. After that, given a specific resource, we prompt the LLM to identify all associated resource references.

To mitigate hallucinations, the initial output undergoes a rigorous self-reflection validation process. We instruct the LLM to evaluate its own response against the predefined conceptual framework established through few-shot examples. Based on this self-assessment, we ask the LLM to make binary Pass/Fail decisions for identified references, eliminating ambiguity in validation outcomes. For failed references, we ask LLM to re-generate the results until self-reflection passes, effectively filtering out incorrect references to improve accuracy.

For instance, when analyzing the resource references of a primary resource Pod, we ask the LLM to act as a Kubernetes expert and provide the LLM with examples from the first two columns of Table 1 for few-shot learning. After that, we prompt GPT-4.5 with the Pod resource and ask it to identify dependent resources. GPT-4.5 initially outputs potential dependent resources (e.g., ServiceAccount, Secrets, Scheduler). Then, through self-reflection and validation prompts, the LLM removes incorrect resources (e.g., Scheduler). Similarly, for the sub-resource replicationcontrollers/scale, we introduce the concept and five sub-resource cases from columns 4 and 5 of Table 1. We ask the GPT-4.5 to build resource reference analysis and make a binary pass/fail self-reflection. GPT-4.5 identifies that replicationcontrollers/scale refers to the Pod resource with a self-reflection pass. Using this method, we systematically collect all resource references for two types of resources. Detailed Q&A steps and results of all resources are posted in our open-source link.

Permission references analysis (LLM): Just as we discussed in §3.3, explicit permissions represent actions of resources that imply consequential operations on other resources, which semantically reveal corresponding implicit permissions beyond the explicit scope. Based on this, we infer implicit permissions by reasoning over explicit permissions within the context of previously derived resource references. We first define permission references using primary and sub-resource relations, supported by examples listed in Table 1. After that, we input the LLM explicit permissions of one resource and direct it to conduct a detailed permission reference analysis for each explicit permission, guided by resource references from stage 1.

To ensure reliability in permission inference, we apply the same rigorous validation process as in stage 1, making the LLM perform self-reflection and revise its output. This sequential application of self-reflection across both stages prevents cascading hallucinations, where errors from resource reference analysis could propagate into permission inference, thereby maintaining accuracy throughout the entire inference pipeline.

For example, for the primary resource Pod, we provide GPT-4.5 with its explicit permissions and ask it to identify whether these operations imply implicit permissions on other referenced resources. These referenced resources (e.g., Secrets) have already been derived in the previous stage. In Kubernetes, the token of ServiceAccount specified by a Pod is stored as a Secret resource. When creating a Pod with a specified Secret, Kubernetes retrieves and mounts the Secret. GPT-4.5 can semantically infer this behavior using its domain knowledge. As a result, GPT-4.5 successfully identifies the permission reference: create Pod → get Secrets. Similarly, for the sub-resource replicationcontrollers/scale, GPT-4.5 shows that the update and patch operations imply implicit permissions such as create/delete Pod. Using this method, we systematically collect implicit permission references for both primary and sub-resources.

4.5.2 Trace-based static analysis: As discussed previously, we employ LLM-based reasoning and static analysis in parallel. While the LLM is well-suited for semantic reasoning, static analysis focuses on rule-based detection of well-defined patterns observed in Kubernetes source code. We designed and implemented a static analysis tool that mirrors the methodology of both resource reference and

Table 2: Summary of analysis results.

Version	Data category	Primary#	Sub#	Total
1.30.10	API resources	70	46	116
	Explicit permissions	504	131	635
	Implicit references	467	126	593
1.29.2	API resources	66	45	111
	Explicit permissions	472	128	600
	Implicit references	467	124	591
1.30.2	API resources	70	46	116
	Explicit permissions	504	131	635
	Implicit references	467	126	593
1.31.2	API resources	69	46	115
	Explicit permissions	496	131	627
	Implicit references	464	126	590

Table 3: Comparison between LLM and static analysis.

Version	Method	Identified#	Triggered#	TR	FNR
1.30.10	LLM+SA	593	328	55.3%	6% (3/50)
	LLM	453	307	67.8%	8% (4/50)
	SA	209	85	40.7%	52% (26/50)
1.29.2	LLM+SA	591	326	55.2%	8% (4/50)
	LLM	451	305	67.6%	10% (5/50)
	SA	208	84	40.4%	54% (27/50)
1.30.2	LLM+SA	593	328	55.3%	6% (3/50)
	LLM	453	307	67.8%	8% (4/50)
	SA	209	85	40.7%	52% (26/50)
1.31.2	LLM+SA	590	330	55.9%	6% (3/50)
	LLM	450	309	68.7%	8% (4/50)
	SA	209	85	40.7%	52% (26/50)

Table 4: Classification of triggered implicit permissions

Version	Critical	High	Medium	Low
1.30.10	61	87	41	139
1.29.2	61	87	41	137
1.30.2	61	87	41	139
1.31.2	61	87	41	141

permission reference analysis. This combined design improves the overall coverage and robustness of permission inference.

Resource references analysis (static): To build resource references via static analysis, our key observation is that Kubernetes controllers monitor and handle verbs of primary resources by invoking handler functions that interact with other resources [8]. In contrast, sub-resource interactions are often tied to HTTP API requests processed by the kube-apiserver. By tracing resource handler functions invoked by controllers or kube-apiserver, we identify resource references, where verbs on one resource may indirectly affect another. At this stage, the exact triggering verbs (e.g., `update`, `patch`) remain unknown and are resolved in the subsequent permission reference analysis.

Permission references analysis (static): Once resource references are established, we analyze which explicit permission verbs (e.g., `update Node`) are monitored and handled by controllers or the kube-apiserver. Specifically, we identify the verbs that trigger the previously traced handler functions, thereby linking explicit verbs to the resulting implicit operations. This two-stage process enables us to construct permission reference relationships, where holding an explicit permission on one resource may imply additional operations on other resources.

For instance, to build the resource reference between a node and other resources, we first trace the handlers called by the node controller, such as `deletePodHandler`. This handler corresponds to pod resources and is triggered when the controller processes node-related events. We build a resource reference (node → pod). After that, we further analyze what specific operations on node resources (e.g., update node) are monitored by the node controller. Based on these two stages, we identify that the `NodeController` invokes `deletePodHandler` when handling a node update event. We can infer that the explicit permission `update Node` leads to the implicit permission `delete Pod`.

4.6 Analysis Results

Evaluation scope: Kubernetes version skew policy maintains the most recent three minor releases (1.29, 1.30, and 1.31 during our experiments). We conducted experiments on versions 1.29.2, 1.30.2, 1.30.10, and 1.31.2, showing consistent patterns that enable our approach to generalize across different Kubernetes releases. The results summarized in Table 2 demonstrate the robustness and version-independence of our methodology.

Evaluation aspects: To evaluate the effectiveness of our approach, we conduct experiments to assess the accuracy of resource identification, explicit permission identification, and implicit permission reference analysis. For the first two aspects, we evaluate the completeness and correctness of our static analysis results. For the third aspect, we use an ablation analysis to quantify the individual contributions of LLM-based reasoning and trace-based static analysis (summarized in Table 3). Building on these results, we perform security risk classification of the triggered implicit permissions to assess their potential security impact (summarized in Table 4). We use another ablation to quantify the two-step self-reflection adopted by LLM analysis (summarized in Table 5). Finally, we discuss the cross-LLM adoption of our methods.

Resource and explicit permission identification: For resource and explicit permission identification, we dynamically query all API Groups in a standard Kubernetes cluster and their corresponding resource/permission lists. To address the potential false negatives caused by non-standard configurations, we cross-checked our resource and permission identification results with official Kubernetes documentation. For example, for version 1.30.10, the most recent branch during our experiments, we successfully cover all resources (98) and explicit permissions (516) present in standard clusters, achieving a 100% recall rate. Additionally, our tool identifies 18 additional resources (116-98) and 119 permissions (635-516) beyond the standard cluster. Through cross-checking the Kubernetes official documents, we confirm that these are related to features available but not enabled in default configurations, rather than misidentification. Resource and permission identification achieve 0% FDRs.

Ablations of LLM and static analysis: For implicit permission analysis, we use a combined approach that consists of an LLM-based reasoning model and trace-based static analysis to analyze implicit permissions. Table 3 shows the comparison results across all tested versions. For 1.30.10, the combined analysis identifies 593 implicit permission references, with the LLM-based approach alone identifying 453 references and the static analysis identifying 209, sharing 69 references between both approaches. These results show the unique value of each method.

Since our work is the first to systematically define and analyze implicit permissions in Kubernetes, no existing ground truth dataset is available for evaluation. To address this, we use the trigger rate and false negative rate to evaluate the result. For the trigger rate, we dynamically trigger the identified implicit permissions in a cluster. For the FNR, we manually construct a ground truth set of implicit permission references from two sources. First, we curate a subset of Kubernetes CVEs that are explicitly related to privilege escalation via permissions. While not all CVEs involve explicit permissions or implicit effects, we selectively include those where privilege escalation can be interpreted through the lens of implicit permission propagation. Second, we incorporate additional references derived from our domain knowledge of Kubernetes resource interactions. The resulting dataset consists of 50 validated references covering a diverse range of privilege escalation patterns. We measure the false negative rate (FNR) as the proportion of these known references that our system fails to identify.

For 1.30.10, the combined system achieves 328 total triggers (55.3% trigger rate) and 6% FNR, outperforming individual approaches: LLM alone (67.8% trigger rate, 8% FNR) and static analysis alone (40.7% trigger rate, 52% FNR). Other versions show similar trades.

The complementary nature is evident: LLM uniquely identified 243 implicit permissions that static analysis missed, while static analysis uniquely identified 21 cases that LLM failed to capture, and the overlap between the two approaches was 64 shared implicit permissions. The LLM excels in cases lacking clear structural patterns or involving complex multi-resource interactions, while static analysis excels in cases with well-defined, repeatable patterns.

Triggered permission classification: The triggered implicit permissions and corresponding explicit ones can be categorized into four levels according to their security impacts. *Critical*, explicit permissions whose implicit counterparts may lead to privilege escalation attacks, as demonstrated by the CVEs discussed in §3.3. *High*, implicit permissions that can lead to denial-of-service attacks by allowing unauthorized resource exhaustion or service disruption (e.g., *delete Namespace* grants *delete all Pods* in that namespace). *Medium*: explicit permissions whose implicit counterparts can lead to sensitive information leakage (e.g., *get Pods/status* may expose additional pod metadata beyond just status information, semantically equal to *get Pods*). *Low*: other explicit permissions whose corresponding implicit permissions lead to less severe risks.

Based on this categorization strategy, we analyzed the implicit permissions across different Kubernetes versions, summarized in Table 4. For example, among the 328 triggered implicit permission references in 1.30.10, 61 critical references, 87 high-risk references, 41 medium-risk references, and 139 low-risk references.

Table 5: Ablation of reflection steps, Ref. means reflection

Version	w/o Ref. 1	w/o Ref. 2	w/o Ref. 1&2
1.30.10	52.9%(14.9%↓)	42.5% (25.3%↓)	27.2% (40.6%↓)
1.29.2	52.9% (14.7%↓)	42.5% (25.1%↓)	27.2% (40.4%↓)
1.30.2	52.9% (14.9%↓)	42.5% (25.3%↓)	27.2% (40.6%↓)
1.31.2	52.5% (16.2%↓)	42.9% (25.8%↓)	27.4% (41.3%↓)

Ablations of LLM hallucination elimination: As discussed in §4.5, we adopted a structured two-step self-reflection mechanism to eliminate LLM hallucinations. To validate the effectiveness of this approach, we conducted ablation experiments by progressively removing hallucination elimination components. The results are summarized in Table 5.

For example, in version 1.30.10, removing the first self-reflection step reduces the trigger rate of the LLM method from 67.8% to 52.9% (370/700), a 14.9% drop in reliable identification. Removing the second step further lowers it to 42.5% (332/782), a 25.3% decrease. Eliminating both self-reflection mechanisms results in only 27.2% trigger rate (487/1793), a 40.6% degradation. The progressive performance degradation validates the contribution of each self-reflection to the overall system reliability. The detailed results are also open-sourced in our repository.

Cross LLMs: Our approach is not limited to GPTs. Instead, it can be adopted with various LLMs. We evaluate several LLMs and select the most suitable one based on performance. Across multiple Kubernetes versions (1.29.2, 1.30.2, 1.30.10, and 1.31.2), we conducted experiments using DeepSeek-R1 and Gemini-2.0-Pro. DeepSeek-R1 achieved stable trigger rates of 61.5%/61.3%/61.3%/60.8% across versions respectively (217/353, 220/359, 220/359, 220/362 triggered references). Gemini-2.0-Pro maintained trigger rates of 51.3%/51.1%/51.1%/50.7% respectively (253/493, 256/501, 256/501, 256/505 triggered references). In comparison, GPT-4.5 achieved superior performance with higher trigger rates across all tested versions.

5 Practical Implicit Permission Attacks

5.1 Ethical Considerations

For ethical considerations, we intend to evaluate the attacks without causing real-world impact. Further, all potential risks should be responsibly reported to the relevant communities and vendors through proper channels. We only discuss acknowledged ones.

For the environment setup, we build a separate cluster locally for CNCF projects. We install applications strictly via their official deployment guides to identify explicit permissions with implicit risks used by them. For cloud environments, we procure dedicated clusters from major vendors, and install all marketplace Kubernetes applications, then analyze implicit permission propagation within cluster boundaries.

For responsible disclosure, we report all findings privately through official vendor and community security channels. For CNCF projects, we only discuss vulnerabilities with assigned CVEs (5 CVEs granted from 56 reported issues). For cloud environments, we only discuss risks officially confirmed by vendors (Google: 5 confirmed out of 31 cases; Azure: 1 confirmed out of 5 cases). Unconfirmed risks in

Table 6: CVE instances demonstrating implicit permission risks

App name	CVE-ID	Explicit permission	Implicit permission	Severity
Kanister	CVE-2024-43403	create/patch/update DaemonSet	get secrets	high
Capsule	CVE-2024-39690	patch namespace	* verbs of * resources	high
Openelb	CVE-2024-41814	create/update/patch DaemonSet&Deployment	get secrets	moderate
OCM	CVE-2024-9779	create Pod	get secrets	moderate
External-secrets	CVE-2024-45041	get/list secrets	* verbs of * resources	moderate

other environments, including unresolved CNCF issues and applications from Amazon AWS or Alibaba Cloud, will remain undisclosed pending CVE assignment or vendor acknowledgment.

5.2 Manual Efforts to Uncover Vulnerabilities

While our automated analysis successfully identifies implicit permission references, converting these theoretical findings into validated security vulnerabilities requires substantial manual efforts. First, not all implicit permission references correspond to explicit permissions that are actually deployed and utilized in real-world environments. Second, transforming implicit permission mappings into exploitable attack vectors requires domain expertise and security knowledge to construct practical proof-of-concept attacks. Finally, ethical considerations mandate that we conduct testing in isolated environments to avoid disrupting other tenants' workloads and responsibly disclose only confirmed vulnerabilities to prevent malicious exploitation.

Based on these insights, we perform a three-step manual process to transform theoretical findings into real vulnerabilities. First, for each deployed application, we extract its RBAC configurations to identify which explicit permissions are granted. We then cross-reference these permissions against our triggered implicit permission mappings to pinpoint applications with potential attack vectors. Second, for those apps with sensitive permissions, we manually construct attack scripts and execute real attacks in separate clusters to demonstrate privilege escalations within controlled environments. Third, we report triggered vulnerabilities through appropriate security channels. We only discuss acknowledged cases, while unconfirmed risks remain undisclosed to prevent malicious exploitation.

While we triggered multiple implicit permissions, only 5 public CVEs are assigned. This discrepancy stems from three factors. First, not all explicit permissions that carry implicit permission risks are actively used in real-world applications. Second, project maintainers typically assign CVEs only for critical security impacts (e.g., privilege escalations). Besides, developers sometimes prioritize functional stability over addressing implicit permission risks. For instance, we report to OpenKruise that the Create Pod explicit permission may carry risks, but they note that removing it would disrupt key functionalities. This underscores the complexity and challenges of solving this problem.

5.3 Real Vulnerabilities in CNCF Projects

In the local environment, CNCF project developers confirmed five implicit permission vulnerabilities, summarized in Table 6.

A potential risk of Capsule: Capsule is a CNCF sandbox project that implements a multi-tenant and policy-based environment in the Kubernetes cluster. Capsule is granted an explicit permission `patch namespace`, and it carries an implicit permission with `* verbs of * resources`. A malicious user can abuse this explicit permission to modify the *metadata.ownerReferences* of that namespace, and he/she can designate themselves as the "owner" of the namespace. After that, the Capsule controller allocates permissions based on owner preferences without thorough verification; the malicious user could illegitimately gain administrative rights of all resources inside that Kubernetes namespace. We reported this issue to its developers, and they awarded us a high-severity CVE.

A potential risk of Openelb: Openelb is a CNCF sandbox project that implements a load balancer for Kubernetes. It has an openelb-controller Deployment, which is bound to the same-name ClusterRole. This ClusterRole has create/patch/update verbs of daemonsets/deployments resources. A malicious user can create a new DaemonSet or Deployment and mount a high-privileged service account in the Cluster (e.g., the cluster-admin account). After that, the attacker can access the newly created Pod and abuse this high-privilege token to execute high-privileged operations on the whole cluster, resulting in a cluster-level privilege escalation. Openelb developers gave us a moderate CVE number.

A potential risk of OCM: OCM is a CNCF sandbox project developed by Red Hat. OCM has a Deployment named cluster-manager, which is bound to a same-name ClusterRole. This ClusterRole has the permission to create Pod resources. Similar to other attacks, an attacker can create a Pod and mount a high-privileged service account, resulting in a cluster-level privilege escalation. The developers confirmed our findings and gave us a moderate CVE.

A potential risk of External-secrets: External-secrets is another CNCF sandbox project maintained by Red Hat. Its Deployment is granted explicit get/list secrets permission. These explicit permissions carry implicit ones for * verbs of * resources. A malicious user can abuse the implicit permission to get high-privileged secrets and perform privilege escalation. Red Hat confirmed our reports and assigned us a moderate CVE.

5.4 Potential Risks in Cloud Environments

Potential risks of Google apps: Google has confirmed 5 apps (i.e., Kubecost, Cert-manager, Metrics-server, Gmp Operator, and Prometheus Operator) from its Marketplace with a security bounty.

Kubecost is a tool for monitoring Kubernetes costs [46]. Google GKE introduces a critical component, kubecost-deployer, as a deployment tool for setting up other components of Kubecost. It is granted "*" verbs of "ClusterRoles" and "ClusterRoleBindings" explicit permissions. As shown in Figure 7, the attacker can create a new ClusterRoleBinding (❶) and bind their own service account to a high-privileged ClusterRole (❷), such as `cluster-admin`. As

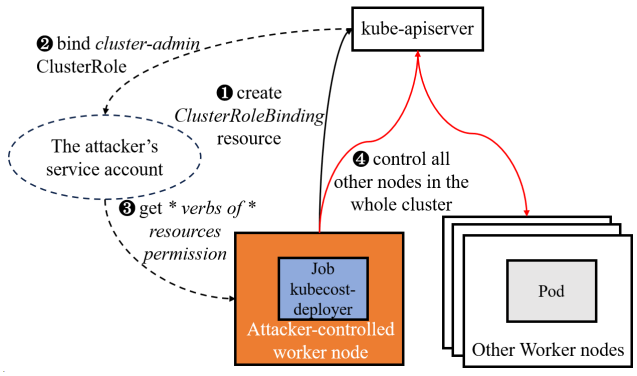


Figure 7: Implicit permission attack via Kubecost.

a result, the bind operation is semantically equivalent to directly granting the attacker's service account implicit permission * verbs of * resources (❸) and taking over the whole cluster (❹).

Cert-manager, an app from the marketplace, has a Deployment named cert-manager. It has the `create pod` explicit permission, which carries an implicit permission with the get verb of the secret resource. If the attacker can create a pod with a high-privileged ServiceAccount, they can get the ServiceAccount's token and make a cluster-level privilege escalation.

Metrics-server is a component of Google GKE metrics collection [29]. It has a `patch deployment` explicit permission, and it carries an implicit one to update the pod managed by the deployment. A malicious user can patch a specific deployment (e.g., metrics-server-v0.5.2) to mount a high-privileged ServiceAccount (e.g., clusterrole-aggregation-controller) into their managed Pod. Then, the high-privileged permissions of this ServiceAccount (e.g., escalate ClusterRole) can be abused to take over the whole cluster.

The Gmp Operator is a component of Google Cloud Managed Service for Prometheus [31]. It has the patch verb for MutatingWebhookConfigurations, and carries implicit permissions to monitor and modify other resources. A malicious user can modify any existing MutatingWebhookConfigurations in the GKE cluster. Consequently, they could monitor and manipulate any events and resources within the cluster by modifying these configurations, leading to potential cluster-level privilege escalation.

The Prometheus Operator has one deployment named prometheus-operator. The ServiceAccount of prometheus-operator has * verbs of StatefulSet resource [4]. This explicit permission carries out the implicit permission to `create` multiple Pods managed by the StatefulSet. The attacker can first create multiple malicious Pods with a high-privileged ServiceAccount. Second, the attacker can get the ServiceAccount's token and make a potential privilege escalation.

We submitted these issues to Google Cloud through the Google Bug Hunter program [28], which classified these issues as *"bypass of significant security controls."* Furthermore, Google announced: *"Our Vulnerability Reward Program panel has decided to issue a reward of $3133.70 for your report."*

Potential risks of Azure apps: The tigera-operator is an app from the Azure Marketplace [24]. This app includes a same-name deployment. It has create, get, and delete verbs on * resources. This configuration allows attackers to potentially exploit these explicit permissions (e.g., create Pods) to launch various types of implicit permission attacks similar to the OCM we discussed before. Microsoft Azure Security Response Center (ASRC) confirmed our findings and said: *"We will be acknowledging you in our Online Service Acknowledgement Portal using the details you submitted in our researcher portal."*

6 Discussions

6.1 Mitigation Discussion

For app developers: As we discussed in §4.6, explicit permission can be categorized into four distinct levels based on their implicit consequences. App developers can avoid using critical and high-risk explicit permissions to minimize security threats. To facilitate practical adoption, we open-source the detailed classification results across all tested Kubernetes versions. This allows developers to cross-reference their applications' RBAC configurations against our risk assessments and make security-conscious permission choices.

For Kubernetes developers: For Kubernetes developers, we propose two system-level solutions: Secret-level protection and permission decoupling for non-Secret resources.

Secrets in Kubernetes bear disproportionate security risks. More than half of the vulnerabilities in Table 6 are related to accessing "Secrets." Consequently, addressing Secret vulnerabilities can significantly reduce the implicit permission attack surface. Specifically, developers can implement encrypted storage for Secrets with decryption keys distributed only to authorized Pods. This approach ensures that Pods with legitimate access requirements can operate normally, while malicious Pods, even with 'get Secret' permissions, can only obtain encrypted data that cannot be misused. A recent work by KubeKeeper [52] demonstrates the feasibility of such fine-grained encryption-based protection in Kubernetes.

For non-Secret resources, developers can decouple implicit permissions from explicit permissions and require all operations to be explicitly authorized before execution. For example, regarding the implicit Secret mounting capability (i.e., `get Secret`) that comes with `create Pod` permission, developers can require users to possess not only `create Pod` but also specific `get Secret` permissions for targeted Secrets. This approach mitigates the cross-resource operational capabilities arising from resource dependencies.

6.2 Limitation Discussion

Justify static analysis over dynamic analysis: Our approach adopts a hybrid analysis-trigger approach that combines static analysis with dynamic trigger verification. This is because our strategy is to conservatively identify all possible implicit permissions through static analysis, then use a dynamic trigger for verification.

In implicit permission discovery, a pure dynamic analysis approach offers the advantage of yielding only authentic, triggerable results with minimal false positives. However, due to the complex triggering contexts, it is challenging to construct meaningful test scenarios without a deep understanding of Kubernetes' internal controller logic and resource dependencies. Our hybrid approach leverages the coverage benefits of static analysis while using dynamic verification to confirm the practical exploitability of identified relationships.

To facilitate further research, we have open-sourced our successfully triggered cases in different versions to support community improvement. Future work could use these verified results as fuzzing seeds to develop adaptive strategies that better understand domain-specific triggering requirements and achieve higher coverage in dynamic analysis.

Omission of implicit permissions: The trigger rate of 55.3% (328/593) primarily stems from verification complexity rather than detection inaccuracy. The untriggered permissions do not necessarily indicate false positives but rather reflect the inherent challenges in the dynamic validation of implicit permissions.

More specifically, triggering implicit permissions requires complex runtime conditions that are difficult to replicate comprehensively in testing environments, including specific cluster configurations, intricate prerequisite combinations, and deep permission chain dependencies. Successfully exploiting these permissions demands significant domain expertise to navigate these complex conditions, which explains the relatively low trigger rate observed in our evaluation. Consequently, these untriggered permissions likely represent real security risks that are challenging to demonstrate, rather than detection errors. To enhance transparency, we have open-sourced triggering results from different versions and the ground truth, enabling independent validation and community-driven improvements to better understand and address complex triggering scenarios.

Hybrid design synergy: Our current analysis takes the union of LLM and static analysis results on comprehensive discovery. We acknowledge that iterative refinement and cross-validation represent promising directions for improving precision. Based on our detailed prompts, researchers can design cross-validation mechanisms between LLM and static analysis results, or develop iterative refinement processes that progressively improve detection accuracy through multiple analysis rounds.

7 Related Work

Permission side-effects: The side effects of granted permissions are a well-known problem. Grace et al. propose that several privileged permissions used by popular Android images can be exposed and extended to other apps [42]. Wu et al. analyze that the excessive permissions of Android preloaded apps can be abused to mount permission re-delegation attacks [56]. Gorski et al. identify the redelegation paths of permission [41] in Android system services [41]. Li et al. demystify that Android custom permissions can be exploited to obtain dangerous system permissions [48]. Lee et al. analyze how adversaries can manipulate discretionary access control policies in Android to grant new permissions to themselves or victims, exposing potential attack paths [47]. However, these studies primarily focus on vulnerabilities allowing existing explicit permissions to indirectly grant additional explicit ones and ignore the risks carried by implicit permissions.

From the defense aspect, access control is extended by information flow control to allow reasoning about end-to-end security guarantees in a system. More specifically, existing IFC tools, such as Valve [37], WatchTower [1], AutoArmor [50], and PolyCruise [49], focus on preventing unauthorized information flow by ensuring that sensitive information only flows within authorized boundaries. Yet, implicit permissions are consequences of authorized explicit operations and do not neatly fit within IFC's notion of unauthorized information flows. The implicit permissions are beyond the defense scope of IFC tools.

Kubernetes security: In the field of Kubernetes security, several works have been proposed to analyze explicit permissions. Yang et al. discuss how to leverage the excessive permissions of Kubernetes third-party apps and make privilege escalations in Kubernetes clusters [57]. Gu et al. further discuss the same problem with a new detection tool [43]. Yuval et al. show that powerful system pods can be abused to make cluster-level privilege escalation [21]. Palo Alto Networks publishes a report to show that excessive permissions can be abused to take over the whole cluster [51]. However, all these papers are case-specific and target excessive explicit permissions. Besides, famous open-source tools such as Checkov [25], rbac-tool [34], krane [32], teerscan [35], kube-linter [33], and trivy [36] are also published to statically analyze the dangerous permissions in applications based on developer-defined sets of dangerous permissions. However, they also focus on identifying dangerous explicit permissions, not targeting the implicit permissions we proposed. Besides, these works only produce vague consequence descriptions for dangerous permissions rather than providing precise, quantitative analyses of their security impacts.

8 Conclusion

In this paper, we reveal a new attack surface in Kubernetes caused by implicit permissions, which emerge from complex interdependencies among resources and permissions. These implicit permissions can be exploited to perform unintended actions beyond explicitly granted permissions, posing significant security risks. To systematically identify implicit permissions, we propose an approach that combines static analysis with large language model capabilities. Our approach successfully identifies 593 implicit permission references, 328 of which can be successfully triggered. We use these results as insights to identify risks in CNCF projects and commercial Kubernetes environments from cloud vendors. The five CVEs, the six acknowledgments, and a $3133.70 security bounty from Google highlight the practical impact of our work.

Acknowledgments

The authors would like to thank our reviewers for their insightful comments. Those comments helped to reshape this paper. This work was partially supported by the Key Research and Development Program of Shaanxi (Program No. 2025CY-YBXM-066), and partially by the '111 Center' (B16037).

References

[1] Kalev Alpernas, Aurojit Panda, Leonid Ryzhyk, and Mooly Sagiv. 2021. Cloud-scale runtime verification of serverless applications. In *Proceedings of the ACM Symposium on Cloud Computing*. 92–107.
[2] Kubernetes Authors. 2022. Kubernetes. https://kubernetes.io/.
[3] Kubernetes Authors. 2023. Deployment. https://kubernetes.io/docs/concepts/workloads/controllers/deployment/.
[4] Kubernetes Authors. 2023. StatefulSets. https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/.
[5] Kubernetes Authors. 2025. API Overview. https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.29/.
[6] Kubernetes Authors. 2025. API Overview. https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.30/.

[7] Kubernetes Authors. 2025. API Overview. https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.31/.

[8] Kubernetes Authors. 2025. Controllers. https://kubernetes.io/docs/concepts/architecture/controller/.

[9] Kubernetes Authors. 2025. Create Code. https://github.com/kubernetes/kubernetes/blob/master/staging/src/k8s.io/apiserver/pkg/registry/generic/registry/store.go#L446.

[10] Kubernetes Authors. 2025. DaemonSet v1 apps. https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.30/#daemonset-v1-apps.

[11] Kubernetes Authors. 2025. Delete Code. https://github.com/kubernetes/kubernetes/blob/master/pkg/registry/core/service/storage/storage.go#L79.

[12] Kubernetes Authors. 2025. Get Code. https://github.com/kubernetes/kubernetes/blob/master/staging/src/k8s.io/apiserver/pkg/registry/generic/registry/store.go#L847.

[13] Kubernetes Authors. 2025. Ingress-nginx CVE-2025-1974: What You Need to Know. https://kubernetes.io/blog/2025/03/24/ingress-nginx-cve-2025-1974/.

[14] Kubernetes Authors. 2025. kube-apiserver. https://kubernetes.io/docs/reference/command-line-tools-reference/kube-apiserver/.

[15] Kubernetes Authors. 2025. Kubernetes Source Code. https://github.com/kubernetes/kubernetes/blob/master/staging/src/k8s.io/apiserver/pkg/endpoints/request/requestinfo.go#L177.

[16] Kubernetes Authors. 2025. Node Code. https://github.com/kubernetes/kubernetes/blob/master/pkg/registry/core/rest/storage_core.go#L294.

[17] Kubernetes Authors. 2025. Pod Code. https://github.com/kubernetes/kubernetes/blob/master/pkg/registry/core/rest/storage_core.go#L232.

[18] Kubernetes Authors. 2025. Secrets. https://kubernetes.io/docs/concepts/configuration/secret/.

[19] Kubernetes Authors. 2025. Service Code. https://github.com/kubernetes/kubernetes/blob/master/pkg/registry/core/rest/storage_core.go#L284.

[20] Kubernetes Authors. 2025. Using RBAC Authorization. https://kubernetes.io/docs/reference/access-authn-authz/rbac/.

[21] Yuval Avrahami and Shaul Ben Hai. 2022. Kubernetes Privilege Escalation: Container Escape == Cluster Admin? https://www.blackhat.com/us-22/briefings/schedule/kubernetes-privilege-escalation-container-escape–cluster-admin-26344.

[22] AWS. 2025. What is Amazon EKS? https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html.

[23] Azure. 2025. Azure Kubernetes Service (AKS). https://azure.microsoft.com/en-us/products/kubernetes-service.

[24] MicroSoft Azure. 2025. Install a Kubernetes application. https://portal.azure.com/view/Microsoft_Azure_Marketplace/MarketplaceOffersBlade/subscription/0ef5e6d1-a771-46d2-98e6-33fc6a9e2cbc/searchInitiatedFrom/AksExtensions/fromContext/AKS.

[25] Bridgecrew. 2025. Checkov. https://github.com/bridgecrewio/checkov/tree/main.

[26] Alibaba Cloud. 2025. Alibaba Cloud Container Service for Kubernetes (ACK). https://www.alibabacloud.com/product/kubernetes.

[27] Alibaba Cloud. 2025. Marketplace. https://cs.console.aliyun.com/?spm=5176.9843921.console-base.dcsk.44e74882hd4cbN/next/app-catalog.

[28] Google Cloud. 2025. Bug Hunters. https://bughunters.google.com/.

[29] Google Cloud. 2025. Configure metrics collection. https://cloud.google.com/kubernetes-engine/docs/how-to/configure-metrics.

[30] Google Cloud. 2025. Marketplace. https://console.cloud.google.com/marketplace/browse?filter=solution-type:k8s.

[31] Google Cloud. 2025. Set up managed collection. https://cloud.google.com/stackdriver/docs/managed-prometheus/setup-managed.

[32] GitHub Contributors. 2025. krane. https://github.com/appvia/krane/.

[33] GitHub Contributors. 2025. kube-linter. https://github.com/stackrox/kube-linter/.

[34] GitHub Contributors. 2025. rbac-tool. https://github.com/alcideio/rbac-tool.

[35] GitHub Contributors. 2025. terrascan. https://github.com/tenable/terrascan/tree/master.

[36] GitHub Contributors. 2025. trivy. https://github.com/aquasecurity/trivy/.

[37] Pubali Datta, Prabuddha Kumar, Tristan Morris, Michael Grace, Amir Rahmati, and Adam Bates. 2020. Valve: Securing function workflows on serverless computing platforms. In *Proceedings of The Web Conference 2020*. 939–950.

[38] Cloud Native Computing Foundation. 2025. CNCF 2023 Annual Survey. https://www.cncf.io/reports/cncf-annual-survey-2023/.

[39] Cloud Native Computing Fundation. 2023. GRADUATED AND INCUBATING PROJECTS. https://www.cncf.io/projects/.

[40] Cloud Native Computing Fundation. 2023. SANDBOX PROJECTS. https://www.cncf.io/sandbox-projects/.

[41] Sigmund Albert Gorski III and William Enck. 2019. Arf: identifying re-delegation vulnerabilities in android system services. In *Proceedings of the 12th conference on security and privacy in wireless and mobile networks*. 151–161.

[42] Michael C Grace, Yajin Zhou, Zhi Wang, and Xuxian Jiang. 2012. Systematic detection of capability leaks in stock android smartphones.. In *NDSS*, Vol. 14. 19.

[43] Yue Gu, Xin Tan, Yuan Zhang, Siyan Gao, and Min Yang. 2024. EPScan: Automated Detection of Excessive RBAC Permissions in Kubernetes Applications. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 11–11.

[44] Kanister. 2025. Adopters. https://github.com/kanisterio/kanister/blob/master/ADOPTERS.md.

[45] Krane. 2025. rules. https://github.com/appvia/krane/blob/master/config/rules.yaml.

[46] Kubecost. 2025. Kubecost | Kubernetes cost monitoring and management. https://www.kubecost.com/.

[47] Yu-Tsung Lee, William Enck, Haining Chen, Hayawardh Vijayakumar, Ninghui Li, Zhiyun Qian, Daimeng Wang, Giuseppe Petracca, and Trent Jaeger. 2021. {PolyScope}:{Multi-Policy} Access Control Analysis to Compute Authorized Attack Operations in Android Systems. In *30th USENIX Security Symposium (USENIX Security 21)*. 2579–2596.

[48] Rui Li, Wenrui Diao, Zhou Li, Jianqi Du, and Shanqing Guo. 2021. Android custom permissions demystified: From privilege escalation to design shortcomings. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 70–86.

[49] Wen Li, Jiang Ming, Xiapu Luo, and Haipeng Cai. 2022. {PolyCruise}: A {Cross-Language} dynamic information flow analysis. In *31st USENIX Security Symposium (USENIX Security 22)*. 2513–2530.

[50] Xing Li, Yan Chen, Zhiqiang Lin, Xiao Wang, and Jim Hao Chen. 2021. Automatic policy generation for {Inter-Service} access control of microservices. In *30th USENIX Security Symposium (USENIX Security 21)*. 3971–3988.

[51] Palo Alto Networks. 2022. Kubernetes Privilege Escalation: Excessive Permissions in Popular Platforms. https://www.paloaltonetworks.com/resources/whitepapers/kubernetes-privilege-escalation-excessive-permissions-in-popular-platforms.

[52] Maryam Rostamipoor, Aliakbar Sadeghi, and Michalis Polychronakis. 2025. KubeKeeper: Protecting Kubernetes Secrets Against Excessive Permissions. (2025).

[53] Amazon Web Services. 2025. AWS Marketplace. https://aws.amazon.com/marketplace/search/results?ref_=header_nav_dm_eks_add_on&FULFILLMENT_OPTION_TYPE=EKS_ADD_ON&filters=FULFILLMENT_OPTION_TYPE.

[54] Container Security Site. 2025. Container Breakout Vulnerabilities. https://www.container-security.site/attackers/container_breakout_vulnerabilities.html.

[55] Güliz Seray Tuncay, Jingyu Qian, and Carl A Gunter. 2020. See no evil: phishing for permissions with false transparency. In *29th USENIX Security Symposium (USENIX Security 20)*. 415–432.

[56] Lei Wu, Michael Grace, Yajin Zhou, Chiachih Wu, and Xuxian Jiang. 2013. The impact of vendor customizations on android security. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*. 623–634.

[57] Nanzi Yang, Wenbo Shen, Jinku Li, Xunqi Liu, Xin Guo, and Jianfeng Ma. 2023. Take over the whole cluster: Attacking kubernetes via excessive permissions of third-party applications. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 3048–3062.